

System Design Interview: Design YouTube

#117: System Design Interview



NEO KIM

JAN 29, 2026  PAID

Get my system design playbook for FREE on
newsletter signup:

✓ Subscribed

- *Share this post & I'll send you some rewards for the referrals.*
- *Block diagrams created using Eraser.*

Building a video-sharing platform like YouTube is a 'classic' system design interview question.

YouTube processes over 500 hours of content that is uploaded every minute and serves billions of video streams daily.

Unlike static file-sharing services, YouTube transforms every upload into dozens of optimized formats, enabling smooth playback across devices from smartphones to 4K TVs, even on slow networks.



newsletter.systemdesign.one

Here are some core challenges that make YouTube interesting from a system design perspective:

- Scale: Exabyte-level storage with millions of concurrent viewers
- Global reach: Low latency for users worldwide
- Adaptive delivery: Automatic quality adjustment for varying network conditions
- Processing complexity: Each upload becomes 6+ resolutions × hundreds of segments¹
- Read-heavy traffic: This demands aggressive caching

Similar architectural patterns appear in platforms such as Netflix, Twitch, and TikTok, as well as in any platform that serves video at scale.

Let's learn how to design YouTube during a system design interview:

Master of Science in AI Engineering (Partner)



MASTER OF SCIENCE IN AI ENGINEERING

Designed in collaboration with



Eduardo Kassner
Chief Data & AI
Officer,  Microsoft

Tomas Pfister
Head of AI
Research 

**8 Production Grade
Projects + Capstone**

**12 Months | Online |
Starts 2nd Feb**

www.SarasAI.org/MasterAI

AI Engineer is the #1 Fastest-Growing Job on LinkedIn, with salaries typically ranging from \$180,000 to \$350,000.

Co-designed with AI leaders from Microsoft (Eduardo Kassner, Chief Data & AI Officer) and Google (Tomas Pfister, Head of AI Research @ Google Cloud)

12 months | 100% online | No exams

Built for working software engineers ready to land AI Engineer positions:

- Build a GitHub portfolio of 8 AI projects
- Bring Your Own Projects or Ideas (BYOP)
- Live weekly AI masterclasses

Developers from Microsoft, Apple, AWS, and TJ Maxx have already joined the program.

By graduation, you'll have the skills, confidence, and credentials to lead AI initiatives. To command top-tier salaries. To shape what comes next.

Cohort starts February 2.

[Learn more & enroll](#)

Questions to Ask the Interviewer

- Candidate: What scale are we designing for?
- Interviewer: 1 million uploads per day, 100 million DAU²
- Why it matters: Scale drives sharding, partitioning, and caching³
- Interview insight: You show you understand capacity planning

===

- Candidate: What's the read-to-write ratio?
- Interviewer: About 100 views for every 1 upload
- Why this matters: It's a read-heavy system, so the design favors streaming
- Interview insight: You show awareness of workload patterns

===

- Candidate: What's the maximum video file size?
- Interviewer: 256 GB
- Why this matters: Forces you to use multipart uploads and chunking⁴
- Interview insight: You consider edge cases

===

- Candidate: What video formats and resolutions do we support?

- Interviewer: MP4, AVI, MOV, and from 240p to 4K⁵
- Why this matters: Needs a strong transcoding pipeline with many output renditions⁶
- Interview insight: Shows attention to real-world constraints

===

- Candidate: What's the target latency for streaming?
- Interviewer: First frame in under 500ms
- Why this matters: Requires CDN⁷ and adaptive streaming⁸
- Interview insight: You focus on user experience

===

- Candidate: What's acceptable for upload processing time?
- Interviewer: Around 10 to 30 minutes
- Why this matters: Processing can be asynchronous
- Interview insight: You understand eventual consistency

===

- Candidate: Is eventual consistency fine for new uploads?
- Interviewer: Yes, a short delay is acceptable
- Why this matters: Availability over strict immediacy
- Interview insight: You show CAP theorem⁹ reasoning

===

- Candidate: What's our uptime target?
- Interviewer: 99.9%
- Why this matters: Need redundancy and failover¹⁰
- Interview insight: You think about reliability

===

- Candidate: Do we support livestreaming?
- Interviewer: For this interview, focus on uploads
- Why this matters: Keeps scope controlled
- Interview insight: You avoid feature creep

Onward.

System Requirements

The design must address a massive scale and specific constraints:

Functional Requirements

Upload Videos

- Users upload from web and mobile clients
- System accepts files up to 256 GB
- Resume interrupted uploads without data loss
- Store raw media for the transcoding pipeline¹¹

Stream Videos

- Smooth playback across regions and devices
- Quality adjusts automatically to network speed (adaptive bitrate)
- First frame appears in under 500ms
- Stability in different connection types

Search Videos

- Discover content by querying titles and descriptions

- Return results in manageable chunks (pagination)
- Support high-cardinality searches (millions of videos)

Non-Functional Requirements

Availability

- Service must remain available during failures
- New uploads don't need instant visibility (eventual consistency is okay)

Scalability

- Support 1 million uploads/day
- Handle 100 million daily active users

Large File Support

- Multipart upload with chunk-level retries
- Resume capability for interrupted uploads
- Efficient storage and processing of massive files
- No progress loss on network failures

Low Latency Streaming

- First frame must appear in under 500ms
- Video playback must adapt to varying network speeds
- Buffering should be minimal during normal network conditions

Low Bandwidth Usage

- Automatic quality adjustment for slow networks
- Smaller segment sizes for stable playback
- Bandwidth optimization for developing regions

Let's keep going!

API Design

API Type: REST (Representational State Transfer).

Why: REST¹² is the industry standard for public-facing web services, offering scalability and ease of integration for web and mobile clients.

API Optimizations

Pre-Signed URLs¹³ (Bypass the App Server)

Instead of uploading videos through your API servers, clients receive a temporary, signed S3 URL. This prevents your application servers from becoming the bottleneck. And users upload directly to blob storage.

Pagination

The search and comments endpoints return results in chunks via cursor-based pagination¹⁴. This prevents massive queries from locking down the database.

The client interacts with the system via a RESTful API.

The design follows a strict philosophy: application servers handle lightweight metadata, while heavy video data is offloaded directly to cloud storage and CDNs.¹⁵

1. Upload Initiation

Endpoint: POST /v1/videos

Description: The server reserves a video ID and returns a pre-signed URL, a temporary link that allows the client to upload directly to S3

without touching the app server.

Sample Request:

○ ○ ○

```
POST /v1/videos HTTP/1.1
Host: api.youtube.com
Authorization: Bearer <user_session_token>
Content-Type: application/json
```

```
{
  "title": "My Vacation Vlog",
  "description": "A trip to Hawaii",
  "file_size_bytes": 5368709120,
  "format": "mp4"
}
```

Sample Response:

○ ○ ○

```
{
  "video_id": "vid_12345xyz",
  "upload_url": "https://s3.aws.com/raw-videos/vid_12345xyz?signature=abcde12345",
  "status": "pending_upload",
  "expiration_seconds": 3600
}
```

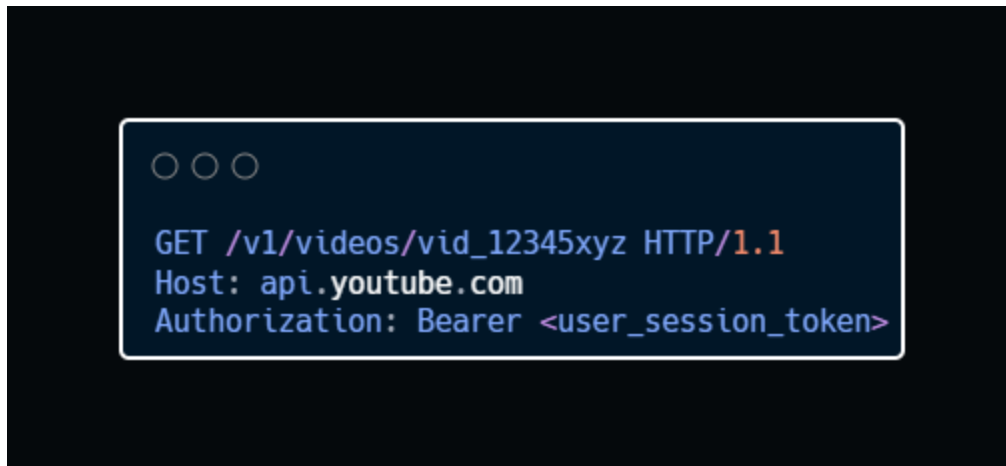
The application server is almost untouched... Bandwidth goes directly to blob storage.

2. Stream Video

Endpoint: GET /v1/videos/{video_id}

Description: When a user requests to watch a video, this endpoint returns the video's metadata and the manifest file URL¹⁶. The client uses this to begin adaptive bitrate streaming (ABR) from the CDN.

Sample Request:



Sample Response:

```
○ ○ ○  
{  
  "video_id": "vid_12345xyz",  
  "title": "My Vacation Vlog",  
  "uploader_id": "user_999",  
  "manifest_url": "https://cdn.youtube.com/vid_12345xyz/master.m3u8",  
  "available_resolutions": ["1080p", "720p", "480p"],  
  "view_count": 10500,  
  "like_count": 340  
}
```

Manifest file tells the client which video segments and resolutions are available. The CDN handles the actual streaming.

3. Update Watch Progress

Endpoint: POST /v1/progress/{video_id}

Description: This endpoint tracks the user's playback position. It's designed for massive scale (handling millions of writes/second) and prioritizes speed over immediate consistency.

Sample Request:

○ ○ ○

```
POST /v1/progress/vid_12345xyz HTTP/1.1
Host: api.youtube.com
Authorization: Bearer <user_session_token>
Content-Type: application/json
```

```
{
  "user_id": "user_123",
  "timestamp_seconds": 125.5,
  "event": "update"
}
```

Sample Response:

○ ○ ○

```
{
  "status": "success"
}
```

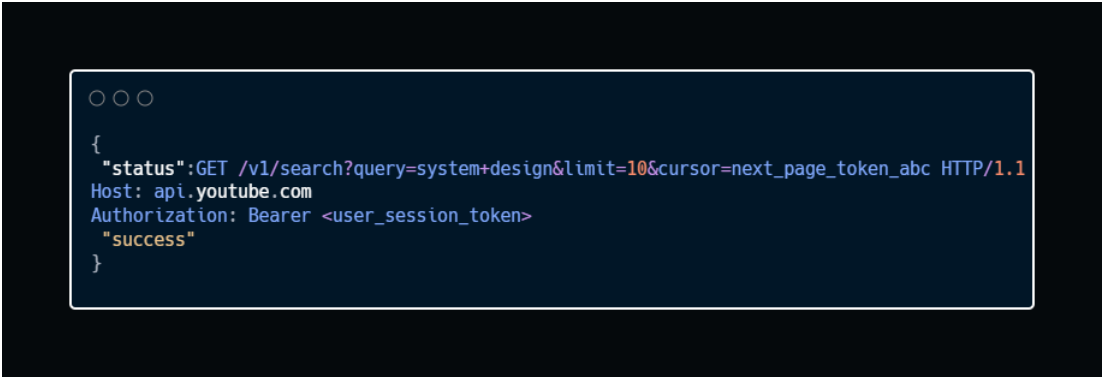
This endpoint alone handles millions of writes/second. It can't afford latency. So fire the request; don't wait for confirmation. Then store in a high-throughput database like DynamoDB¹⁷.

4. Search

Endpoint: GET /v1/search

Description: Queries the search index¹⁸ to find videos matching a user's query. This endpoint relies heavily on pagination to return results in manageable chunks.

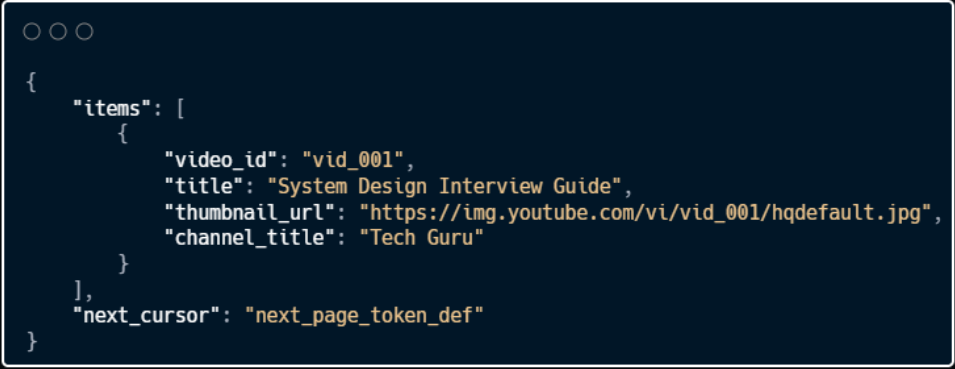
Sample Request:

A sample HTTP request is displayed in a code editor window with a dark blue background and a white border. The request is a GET call to the YouTube API search endpoint. It includes a query parameter for 'system+design', a limit of 10, and a cursor token for pagination. The request also specifies the host as api.youtube.com and includes a Bearer token for authorization. The response body is a simple JSON object indicating success.

```
{}  
{  
  "status": GET /v1/search?query=system+design&limit=10&cursor=next_page_token_abc HTTP/1.1  
  Host: api.youtube.com  
  Authorization: Bearer <user_session_token>  
  "success"  
}
```

Cursor-based pagination prevents massive queries. Each page contains a next_cursor token, allowing the client to fetch the next batch without re-querying the entire dataset.

Sample Response:



```
{}  
  
{  
  "items": [  
    {  
      "video_id": "vid_001",  
      "title": "System Design Interview Guide",  
      "thumbnail_url": "https://img.youtube.com/vi/vid_001/hqdefault.jpg",  
      "channel_title": "Tech Guru"  
    }  
  ],  
  "next_cursor": "next_page_token_def"  
}
```

Cursor-based pagination is more scalable than offset-based pagination. It prevents the “skip 10 million rows” problem¹⁹.

Ready for the best part?

Database Design

The Problem

No single database can handle YouTube’s workload.

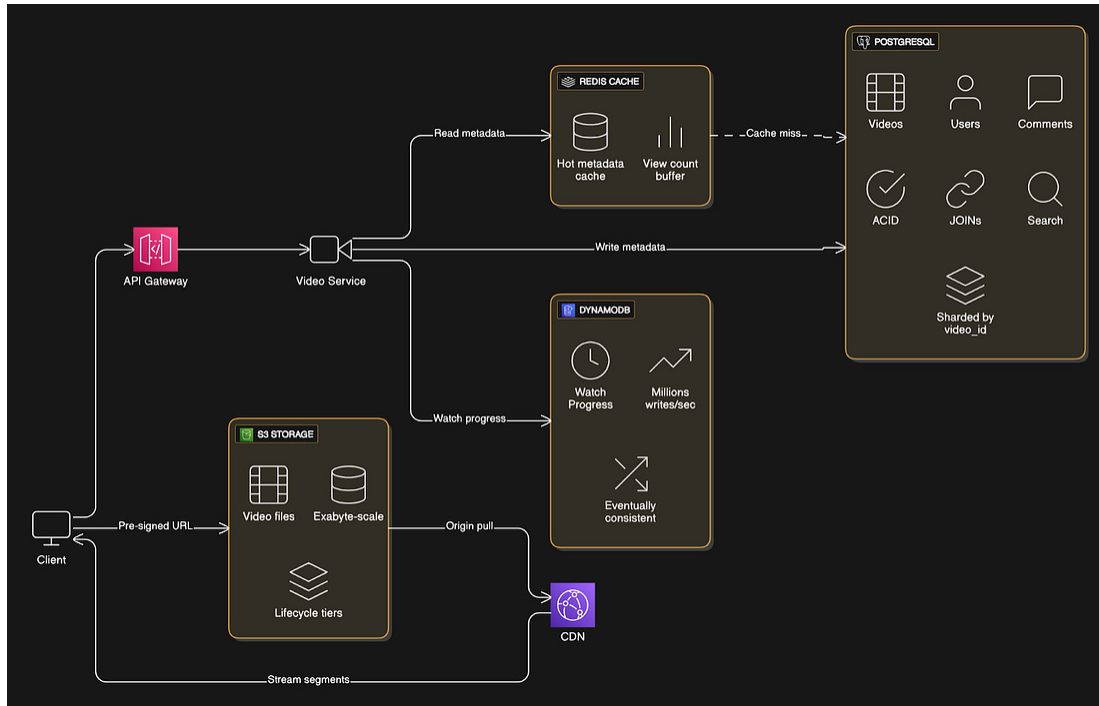
Different features have ‘conflicting’ requirements:

- Video metadata needs ACID transactions and complex queries.
- Watch progress requires millions of writes/second.
- Video files require exabyte-scale storage.
- View counts need microsecond-level caching.

This forces a hybrid data model where each component uses the right tool for the job.

Database Schema Design

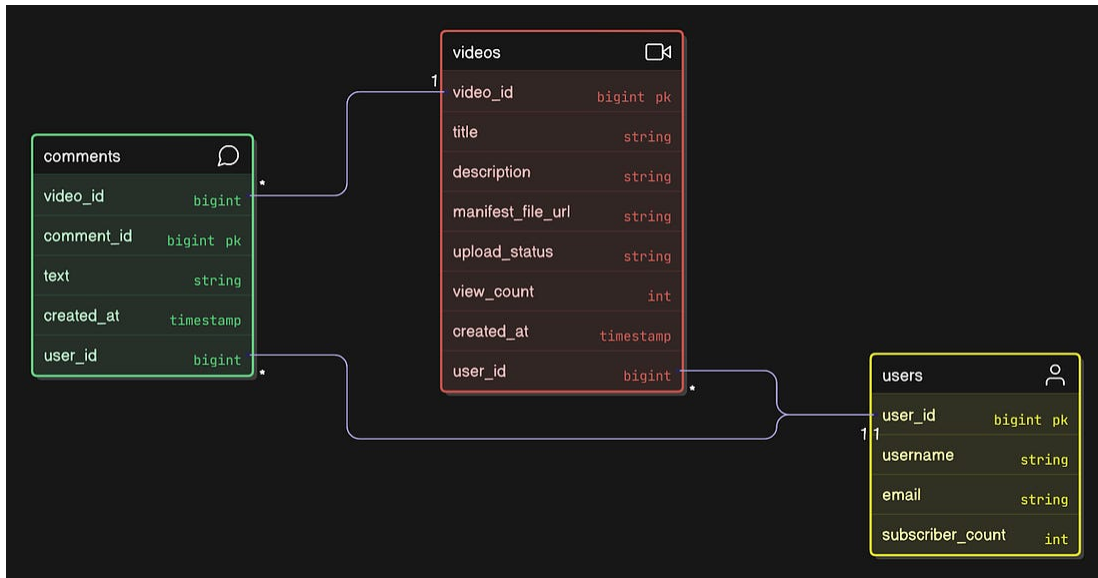
The core architecture separates data into four distinct storage systems.



PostgreSQL

The relational database stores video metadata, users, comments, and subscriptions.

Here are the core tables:



videos table

It stores basic information such as video_id, title, description, duration, and manifest_file_url pointing to the video segments.

To handle reads from 100 million daily users, we shard this table by video_id.

users table

It stores account information such as usernames, email, and subscriber_count.

comments table

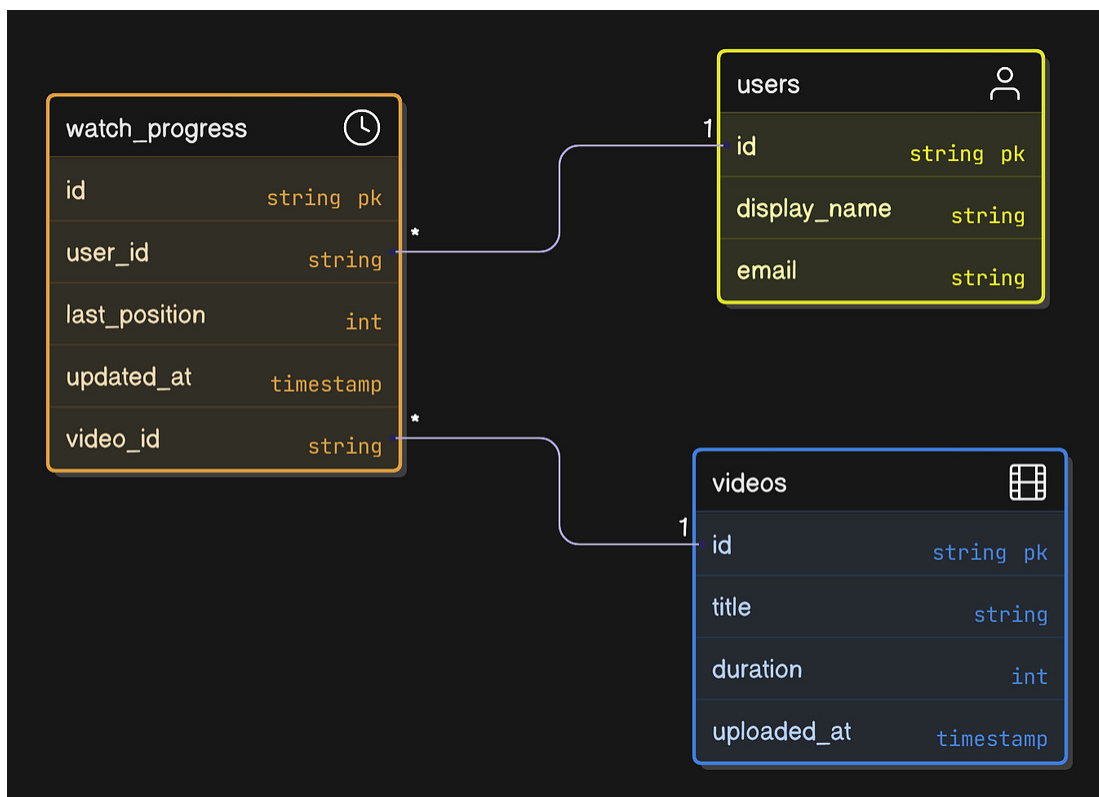
We co-locate this on the same shard as the videos table. This lets us query comments for a video using local JOINS without cross-shard network calls²⁰.

Example video record


```
{}  
  
{  
  "video_id": "abc123",  
  "user_id": "user_456",  
  "title": "How to Code in Python",  
  "manifest_file_url": "s3://bucket/abc123/manifest.m3u8",  
  "upload_status": "ready",  
  "view_count": 50000  
}
```

DynamoDB

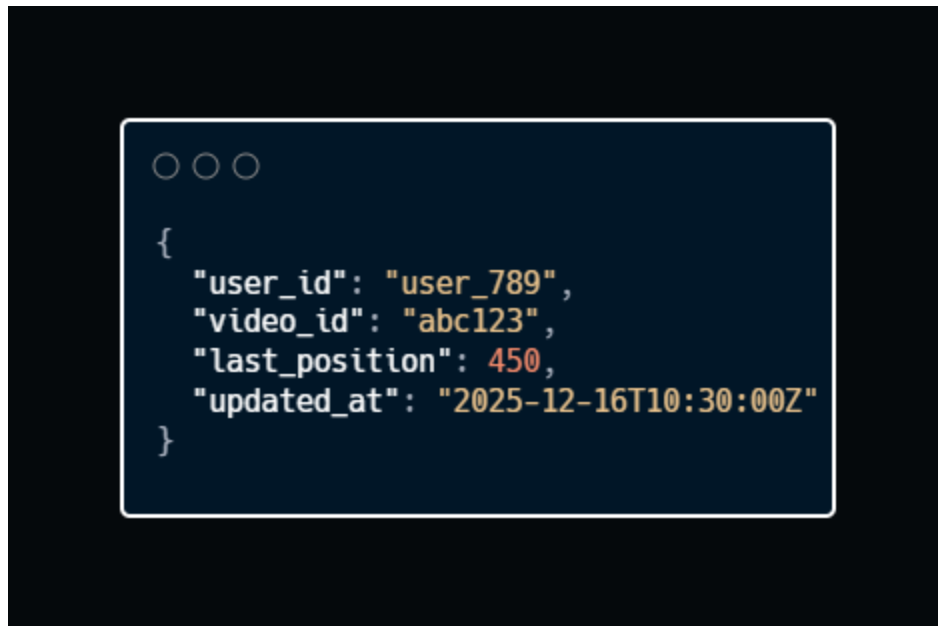
NoSQL database handles watch progress updates that happen every 30 seconds during playback.



watch_progress table

It tracks where users stopped watching. And stores the last_position (timestamp), for users to resume playback. This table lives in DynamoDB because of high throughput requirements.

Example record



Amazon Simple Storage Service (S3)

Blob storage²¹, such as AWS S3, holds all video content:

- Raw uploaded videos
- Transcoded segments (all resolutions)
- Manifest files (.m3u8 playlists²²)
- Thumbnails

Modern blob storage systems scale to exabyte-level capacity while providing exceptional durability through automatic replication across multiple facilities.

Redis

Redis sits in front of the database as an in-memory cache:

- Hot video metadata (title, description, uploader information)
- View count buffers (accumulated before batch writes)
- User session data
- Temporary state for active streams

Redis provides sub-millisecond latency for frequently accessed data.

Cache entries expire after 10 minutes to prevent stale data. The LRU (Least Recently Used) eviction policy automatically removes cold data when memory fills up, ensuring the cache naturally prioritizes the most popular content.

Know someone who wants to learn system design? Consider gifting a subscription:

[Give a gift subscription](#)

Why These Database Choices?

Each data storage is optimized for its specific workload:

PostgreSQL for Metadata

The metadata database serves as the source of truth for all video information.

Here's why PostgreSQL fits:

- ACID transactions ensure correctness:

- Upload status changes must be atomic.
- A video can't show up in search until its status changes from "transcoding" to "ready".
- PostgreSQL guarantees these transitions happen reliably.
- Complex queries with JOINS:
 - Queries frequently need data from multiple tables.
 - Fetching a video with uploader information and top comments requires joining videos, users, and comments tables.
 - PostgreSQL handles these relationships naturally.
- Full-text search capabilities:
 - Users search by title, description, and tags.
 - PostgreSQL provides built-in full-text search that scales reasonably well for metadata queries.
- Mature ecosystem:
 - Proven scaling patterns exist.
 - Monitoring tools, backup solutions, and operational playbooks are battle-tested across millions of deployments.

Tradeoff: PostgreSQL can't handle the massive write throughput required by watch progress tracking.

DynamoDB for Watch Progress

It's difficult to set up a traditional relational database to handle this scale.

Here's why DynamoDB is a good choice:

- Extreme write throughput:
 - Millions of active users update their positions every 30 seconds, generating extreme write throughput.

- Operational simplicity:
 - DynamoDB autoscales²³ without manual sharding configuration or capacity planning.
- Eventual consistency acceptable:
 - Users don't notice if their progress takes a few seconds to save.
 - Rewinding slightly after reopening a video is perfectly acceptable, making eventual consistency a reasonable trade-off.
- Simple access pattern:
 - All queries are get/put operations by (user_id, video_id) composite key.
 - DynamoDB excels at this pattern.

Traditional relational databases require complex sharding and operational overhead to reach this scale.

Tradeoff: no complex queries or JOINS.

S3 for Video Files

Video files dominate storage needs.

Here's why blob storage is a viable option:

- Exabyte-scale capacity:
 - The system stores exabytes annually when accounting for transcoding and replication.
 - Traditional databases and file systems can't scale to this level without massive operational complexity.
- Direct client uploads:

- Pre-signed URLs let clients upload directly to storage without routing through application servers.
- This keeps API servers lightweight and reduces bandwidth costs.
- Automatic tiering:
 - Lifecycle policies move content between storage tiers based on age and access patterns.
 - Frequently accessed videos remain in hot storage with CDN caching, while older content migrates to archive tiers that trade retrieval speed for reduced costs.
- Seamless CDN integration:
 - S3 integrates directly with CloudFront CDN for global distribution.
 - The CDN pulls content from S3 on cache misses and serves it from edge locations for subsequent requests.
- Pay-per-use pricing:
 - No upfront capacity provisioning.
 - Storage costs scale linearly with actual usage.

Tradeoff: Not suitable for low-latency metadata queries. Video files and metadata require separate storage systems.

Redis for Caching

Here's why we need a cache server:

- Sub-millisecond latency:
 - Popular video metadata returns from memory in microseconds instead of milliseconds from the database.
 - This keeps the streaming experience snappy.
- Massive load reduction:

- Caching hot metadata reduces database load by orders of magnitude.
- The database serves only cache misses for less popular content.
- View count buffering:
 - Instead of updating the database on every view, Redis buffers increments.
 - A background job flushes these to PostgreSQL in batches every few minutes, dramatically reducing the write load.
- Automatic eviction:
 - LRU (Least Recently Used) eviction removes cold data when memory fills up.
 - The cache naturally prioritizes hot content without manual intervention.
- Short TTL acceptable:
 - Cache entries expire after 10 minutes.
 - This prevents stale data from persisting too long while still providing massive performance benefits.

Tradeoff: Cache misses still hit the database. The system must handle cold-start scenarios when the cache is empty.

SQL Queries

1. Streaming Video

When a user clicks play, the system fetches video metadata needed for playback and loads uploader information in parallel for the UI:

```
SELECT
  v.video_id,
  v.title,
  v.manifest_file_url,
  v.view_count,
  u.username
FROM videos v
JOIN users u ON v.user_id = u.user_id
WHERE v.video_id = 'abc123'
AND v.upload_status = 'ready';
```

This query hits a single database shard based on `hash(video_id)`.

Single-shard lookup provides fast response times by avoiding cross-shard coordination. The `manifest_file_url` tells the client where to download video segments from the CDN.

2. Tracking Watch Progress

Every 30 seconds during playback, clients fire this update:

```
-- DynamoDB operation
UPDATE watch_progress
SET last_position = 480, updated_at = NOW()
WHERE user_id = 'user_789' AND video_id = 'abc123';
```

The operation is fire-and-forget; clients send the update without waiting for confirmation. This keeps the video player responsive.

3. View Count Updates

View counts use a two-step caching strategy to avoid thrashing²⁴ the database:

```
# Step 1: Increment in Redis (instant)
INCR video:abc123:views

# Step 2: Background job syncs every 5 minutes
UPDATE videos SET view_count = view_count + 150
WHERE video_id = 'abc123';
```

Redis absorbs the real-time increments.

A background process periodically flushes the buffered counts to PostgreSQL in batches. Users accept that view counts might lag by a few minutes because real-time accuracy isn't critical for this metric.

4. Get Top Comments

```
SELECT
  c.comment_id,
  c.text,
  c.likes_count,
  u.username
FROM comments c
JOIN users u ON c.user_id = u.user_id
WHERE c.video_id = 'abc123'
  AND c.parent_comment_id IS NULL
ORDER BY c.likes_count DESC
LIMIT 20;
```

This query efficiently fetches the most popular comments for a video by sorting on likes, which works well at scale when indexed properly.

5. Get User's Uploaded Videos

```
SELECT
  video_id,
  title,
  thumbnail_url,
  view_count,
  created_at
FROM videos
WHERE user_id = 'user_456'
ORDER BY created_at DESC
LIMIT 50;
```

Because the Videos table is sharded by `video_id`, a query filtered by `user_id` cannot be routed to a single shard.

Application server must send parallel queries to all shards and aggregate the results. This scatter-gather²⁵ query is slower but acceptable because it is used infrequently (only in the creator dashboard) and aggregated results are cached for several minutes to avoid expensive repeated queries.

Database Sharding Strategy

Why shard by `video_id`

Most traffic is random video access.

Users click different videos constantly, creating an even access pattern. Sharding by `video_id` distributes the load evenly across all shards. Each video hits exactly one shard, and direct lookups are fast.

If the system were sharded by `user_id` instead, celebrity channels would overload their shard, while new creators would sit idle on underutilized shards. This hotspot problem would create massive performance issues.

Sharding by `video_id` solves this because even viral videos are represented by just one record per shard.

How Sharding Function Works

```
shard_number = SHA256(video_id) % 1000
```

This hash function distributes videos uniformly. Examples:

```
video_id "abc123" → SHA256 hash → mod 1000 → Shard #547
```

```
video_id "xyz789" → SHA256 hash → mod 1000 → Shard #892
```

```
video_id "def456" → SHA256 hash → mod 1000 → Shard #231
```

Re-sharding When We Grow

As data grows, the system requires more shards.

Here's the strategy:

- Add new shards incrementally
 - Add new shards as needed, without requiring a fixed shard growth pattern.
- Use consistent hashing
 - Map both video IDs and shards to positions on the hash ring.
 - When new shards are added, only a limited number of keys get reassigned, minimizing data movement.

- Minimize data movement
 - When adding N new shards to M existing shards, approximately $N / (M + N)$ of the data needs to be migrated, assuming uniform distribution.
 - This is less disruptive than remapping all data.
- Background migration
 - A background rebalancer gradually copies the affected data ranges to the new shards, avoiding load spikes or downtime.
- Zero-downtime migration
 - During the transition, the application server performs dual reads (checking new shards first and falling back to old ones).
 - So queries continue to work while data is being moved.
- Flip traffic once complete
 - After all data is copied and verified, routing gets fully switched to the new shard map, and the old shards are decommissioned.

This approach avoids naive modulo rehashing:

For example, changing ``hashvideo_id %1000`` to ``% 2000``.

...this would remap most keys and require moving nearly all data, causing significant operational disruption.

Ready to do some math?

Know someone who wants to learn system design? Consider gifting a subscription:

[Give a gift subscription](#)

Capacity Planning

Here's the structured approach with sample numbers:

Assumptions

- 100 million DAU (Daily Active Users)
- 1 million uploads/day (1% of users upload)
- Average video length = 10 minutes
- Average raw video size = 500 MB (after compression)
- Read to Write = 100:1
- Transcoding multiplier = 6 resolutions (4K, 1080p, 720p, 480p, 360p, 240p)
- Replication factor = 3

Traffic

Peak hours bring millions of concurrent streams.

Write Operations (Uploads):

- 1 million uploads/day
- Upload QPS = $1,000,000 / 86,400 \text{ seconds} \approx 12 \text{ uploads/second}$

Read Operations (Video Views):

- $100\text{M DAU} \times 100 \text{ views} = 10 \text{ billion views/day}$
- Read QPS = $10,000,000,000 / 86,400 \approx 115,740 \text{ requests/second}$
- Peak QPS (assume 2x average) = $231,000 \text{ requests/second}$

Watch Progress Updates:

- $100\text{M DAU} \times 60 \text{ minutes average watch time} = 6 \text{ billion watch minutes/day}$

- Updates every 30 seconds = 2 updates/minute
- Total updates = $6B \times 2 = 12$ billion updates/day
- Progress update QPS = $12,000,000,000 / 86,400 \approx 139,000$ writes/second
- Peak QPS (2x) = 278,000 writes/second to DynamoDB

Storage

Daily uploads reach petabytes of raw video.

Daily Upload Storage:

- Raw: $1M \text{ videos} \times 500 \text{ MB} = 500 \text{ TB/day}$
- After transcoding (6 resolutions): $500 \text{ TB} \times 2 = 1 \text{ PB/day}$
- With replication (3x): $1 \text{ PB} \times 3 = 3 \text{ PB/day}$

Annual Storage:

- $3 \text{ PB/day} \times 365 = 1,095 \text{ PB/year} \approx 1 \text{ EB/year}$

5-Year Total:

- 5 Exabytes (with automatic tiering to cold storage)

Bandwidth

CDN absorbs over 95% of streaming bandwidth through edge caching.
So origin storage only serves long-tail cache misses.

Upload Bandwidth:

- $500 \text{ TB/day} \div 86,400 \text{ sec} = 5.8 \text{ GB/sec} = 46 \text{ Gbps}$

Streaming Bandwidth (without CDN):

- $10B \text{ views} \times 100 \text{ MB average} = 1 \text{ EB/day}$

- $1 \text{ EB} \div 86,400 \text{ sec} = 11.6 \text{ TB/sec} = 93,000 \text{ Gbps}$

With CDN (95% cache hit rate):

- Origin serves only 5% = 4,650 Gbps
- CDN handles the remaining 88,350 Gbps at the edge

Infrastructure

Redis caches hot metadata. API servers auto-scale with traffic.

Database (PostgreSQL):

- 1,000 shards for video metadata
- 3,000 read replicas for read-heavy traffic

Cache (Redis):

- 100 GB for hot video metadata

Watch Progress (DynamoDB):

- Autoscales to 278,000 writes/sec at peak

Video Storage (S3):

- 5 EB total with tiered storage (hot/warm/cold)

Transcoding Workers:

- 14,000 concurrent workers (elastic spot instances)

CDN:

- 200 edge locations globally
- Caches 500 PB of popular content

API Servers:

- 350 servers (handles 232,000 peak QPS with 3x redundancy)

Know someone who wants to learn system design? Consider gifting a subscription:

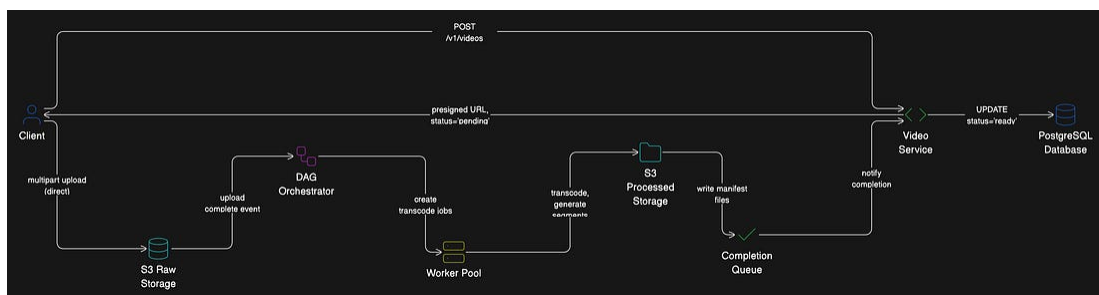
Give a gift subscription

High-Level Design

The architecture decouples the system into two independent pipelines:

- Upload Flow (Write Path), which focuses on reliability and processing correctness.
- Streaming Flow (Read Path), which focuses on low latency and high availability.

Write Workflow



The upload process prioritizes non-blocking operations through these steps:

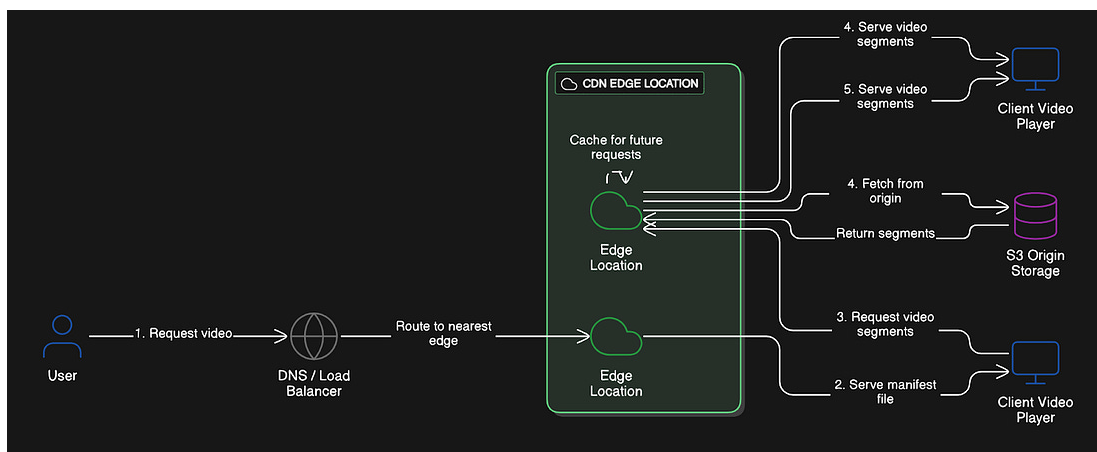
1. Client requests a pre-signed URL from the API.

2. Then, client performs a multipart upload directly to Blob Storage (S3).
3. S3 triggers an asynchronous transcoding pipeline managed by a DAG orchestrator.
4. Workers in this pipeline chunk the video and transcode it into multiple resolutions (4K, 1080p, 720p, 480p, 360p, 240p)
5. Workers save the transcoded results back to Blob Storage
6. A completion message gets sent to a dedicated queue
7. Video service updates the metadata database to mark the video as "Ready."

This architecture is non-blocking because the upload request returns before transcoding completes; the video becomes available once processing finishes.

Async processing means delays are acceptable since users understand videos require processing time. The system achieves eventual consistency; videos become visible once ready.

Read Workflow



Streaming process is optimized for speed through these steps:

1. A user requests a video, and DNS routes them to the nearest CDN Edge Location.
2. Client downloads the manifest file, which lists available segments and resolutions.
3. Client's player dynamically requests video segments from the CDN that match the user's current internet speed, using Adaptive Bitrate Streaming.
4. If content is missing from the edge cache (a "long-tail" video²⁶), the CDN fetches it from the Origin server/Blob Storage.
5. CDN then caches the content for future requests.

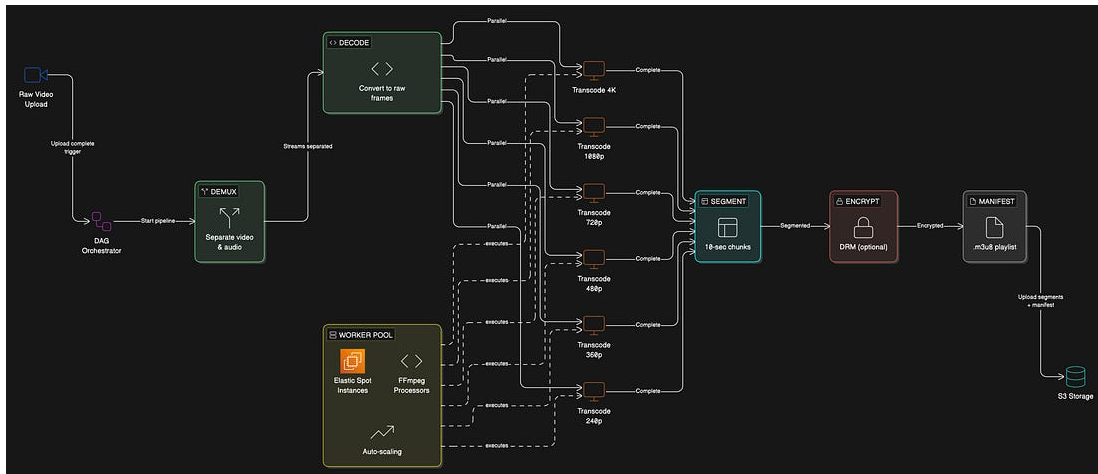
This design optimizes for sub-500ms first frame latency through global edge caching and automatic quality adaptation.

Know someone who wants to learn system design? Consider gifting a subscription:

[Give a gift subscription](#)

Deep Dive & Scaling Considerations

1. Video Processing Pipeline



The transcoding pipeline is where YouTube's complexity lives...

A single uploaded video must become dozens of files across multiple resolutions and formats:

When a user uploads a 4K video, the system generates six resolution versions, ranging from 4K down to 240p. Each version gets split into short segments, creating hundreds of total segments across all resolutions.

Processing this takes time and requires massive computing resources.

DAG-Based Solution

System uses a Directed Acyclic Graph (**DAG**) orchestrator, like Airflow or Step Functions²⁷.

DAGs break processing into discrete tasks that run in sequence and in parallel:

1. First, demux task separates video and audio streams.
2. Next, the decode task converts compressed video to raw frames.
3. Then, multiple transcode tasks generate all resolution versions simultaneously on different workers.

4. After transcoding, the segment task splits each version into short chunks. For premium content, an encryption task applies DRM²⁸ protection.
5. Finally, the manifest task generates playlist files that list all available segments.

Per-Segment Encoding Optimization

Modern transcoding systems use per-segment encoding based on scene complexity.

Each segment analyzes its content characteristics before encoding:

Quiet or static scenes, such as a still landscape or talking head, use a lower bitrate even at the same resolution, while action-heavy scenes (explosions, sports, rapid movement) require a higher bitrate to maintain visual quality.

This optimization reduces storage costs and bandwidth usage without compromising viewers' perceived video quality.

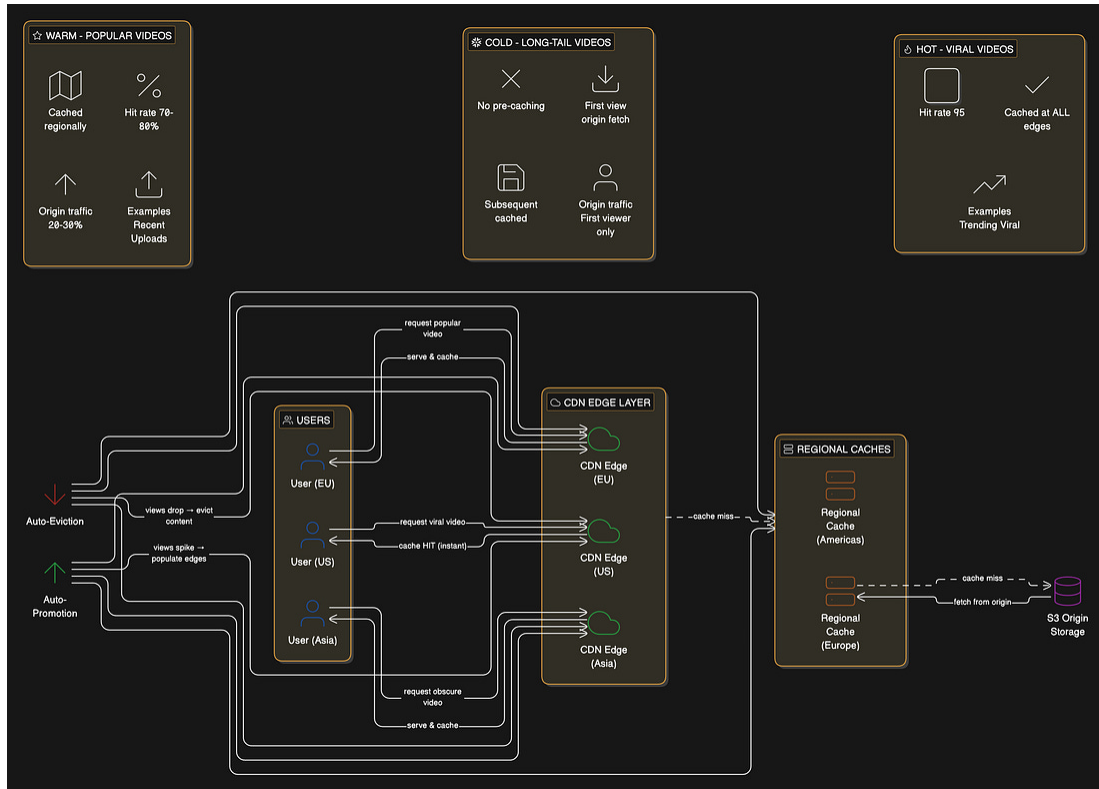
Plus, this architecture provides critical benefits:

- Multiple resolutions transcode in parallel, dramatically reducing total processing time.
- If a worker crashes mid-task, another worker picks up the failed task from the queue.
- Workers checkpoint their progress periodically, enabling them to resume from failures rather than starting over.
- The queue depth automatically triggers worker-pool scaling, handling upload spikes without manual intervention.

The technology stack uses FFmpeg²⁹ for transcoding, spot instances for cheap compute, and S3 for all inputs and outputs.

Spot instances³⁰ cost a fraction of on-demand pricing, and transcoding jobs tolerate interruption since failed tasks simply retry.

2. CDN and Caching Strategy



The CDN is YouTube's most critical infrastructure...

Without it, streaming would be impossible at scale. Daily video views generate petabytes of bandwidth that would cost enormous amounts to serve from origin storage and create terrible latency for distant users.

Multi-Tier Caching Architecture

CDN implements multi-tier caching that adapts to viewing patterns.

Viral videos cache at every edge location globally, achieving very high cache hit³¹ rates with minimal origin traffic. Popular videos get cached regionally, with some origin requests for cache misses³². Long-tail

videos don't pre-cache anywhere; the first viewer in a region triggers cache population from the origin, but subsequent viewers hit the cached copy.

CDN automatically promotes and demotes content based on viewing patterns.

When views spike, edge caches populate. When views drop, edge caches evict the content to make room for newer trending videos. This dynamic tiering optimizes cache utilization without manual intervention and saves enormous bandwidth costs by keeping hot content close to users.

ISP-Level Caching & Access Network Integration

CDNs are not just public cloud edge servers distributed globally.

YouTube also embeds caching infrastructure directly within Internet Service Provider (**ISP**) networks and at Internet Exchange Points (**IXPs**)³³. These ISP-level caches sit physically closer to end users than traditional CDN edge locations, often within the same network infrastructure that delivers internet to homes and businesses. This drastically reduces latency and eliminates the cost of inter-network data transfer.

Content never leaves the ISP's network, providing faster delivery and lower costs for both YouTube and the ISP.

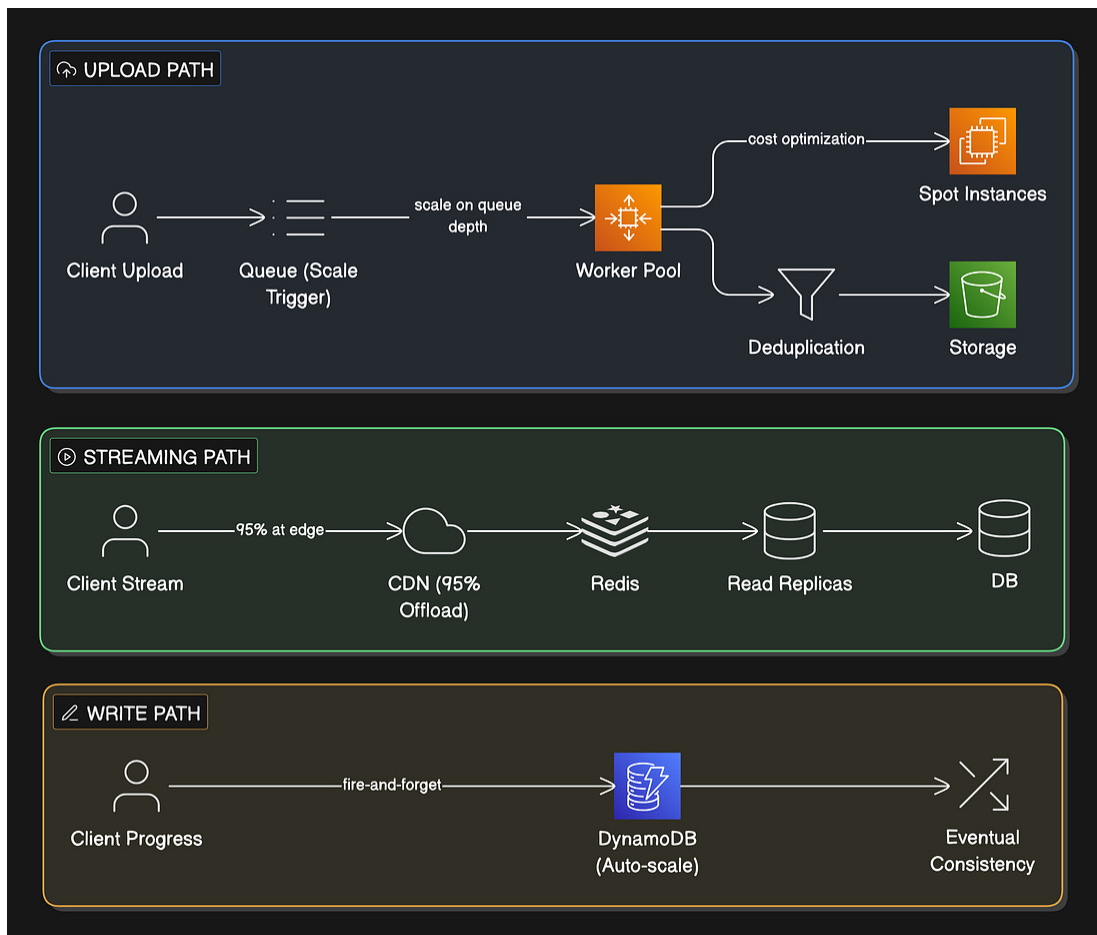
Cache Pre-Warming

For predictable demand spikes, such as a highly anticipated product launch video or a live event recording, YouTube uses cache-prewarming strategies.

i.e., system proactively pushes content to edge locations and ISP caches before users request it.

This ensures the first viewers experience the same low latency as subsequent viewers, avoiding the “cold start” problem³⁴, where initial requests must fetch from origin storage.

3. Handling Scale



Upload Path Scaling

The bottleneck here is transcoding compute,,, not storage.

System uses elastic worker pools that scale from hundreds to thousands based on queue depth. When uploads spike, new workers get provisioned automatically. When the queue empties, workers are terminated to reduce costs.

System uses spot instances for transcoding because they cost substantially less than standard compute. Transcoding tolerates interruption since failed jobs simply retry. System also deduplicates uploads by hashing video content.

If someone uploads content that already exists, the system skips transcoding and links to existing files.

Streaming Path Scaling

The bottleneck here is bandwidth... not compute.

CDN offloads the vast majority of traffic from the origin, with only a small percentage of requests hitting the origin storage for cache misses. Redis caches hot metadata in memory, drastically reducing database load. Video metadata queries hit Redis first, only falling back to the database on cache misses.

The metadata database uses multiple read replicas to distribute read-heavy traffic across many servers.

Write Path Scaling

Watch progress tracking generates millions of writes per second from active users.

DynamoDB handles this extreme write load through auto-scaling without manual intervention. Clients fire progress updates without waiting for confirmation, prioritizing speed over immediate consistency. Users accept the eventual consistency of this feature since rewinding for a few seconds is acceptable.

Traditional relational databases would require complex sharding and massive operational overhead to achieve this throughput.

Failure Handling

System handles failures at every layer:

- Heartbeat-based³⁵ health monitoring continuously checks service health, with load balancers automatically steering traffic away from unhealthy nodes to maintain availability.
- Upload failures use the multipart protocol with chunk-level retries and exponential backoff³⁶ to prevent the system from being overwhelmed during recovery.
- Transcoding failures leverage queue-based processing that automatically reassigns failed tasks to healthy workers, with workers checkpointing³⁷ their progress for resumption.
- Database failures trigger automated primary-replica failover within seconds while read replicas continue serving queries, keeping the streaming experience unaffected.
- All operations are idempotent, making retries safe and preventing corruption.
- Cross-region replication enables survival of complete regional outages by failing over to a different geographic region.

Security Considerations

Content protection operates at multiple layers...

- All data is encrypted in transit using TLS and at rest in storage.
- Premium content uses DRM systems with license servers that issue time-limited decryption keys.
- Users must prove viewing rights before receiving keys, and keys expire to prevent indefinite access.
- Watermarking embeds identifying marks into video streams to trace pirated content back to the source account.

Access control uses multiple mechanisms:

- Pre-signed URLs provide time-limited, cryptographically signed upload access without exposing storage credentials.

- CDN authentication uses token-based access control for premium content.
 - Rate limiting prevents API abuse by capping request rates per user and implementing progressive backoff for violators.
-

Closing Thoughts

Designing YouTube teaches fundamental lessons about distributed systems.

Uploads tolerate latency, while streaming demands speed, so the architecture separates these paths completely and optimizes each independently.

No single database handles all workloads, so the system uses PostgreSQL for metadata, DynamoDB for high-throughput writes, Redis for caching, and S3 for files. Each tool excels at its specific job.

CDN is non-negotiable at scale.

It handles the vast majority of streaming traffic, making the economics work and keeping latency acceptable for global users. Without aggressive CDN usage, origin bandwidth costs would be prohibitive.

Design for failure at every layer since distributed systems fail constantly. Making operations idempotent, implementing retries with exponential backoff, and assuming components will crash lead to resilient systems.

Async processing enables massive scale.

Users don't need instant transcoding, so decoupling upload from processing allows enormous parallelism. Cache aggressively at every

layer because caching is the most effective optimization at scale. Redis caches metadata, the CDN caches video segments, and clients cache manifests. Each layer reduces load on downstream systems.

These aren't compromises born from limitations... They're deliberate design choices that enable operation at scale that would otherwise be impossible.

Understanding why these choices matter separates good system design from great system design.

Know someone who wants to learn system design? Consider gifting a subscription:

Give a gift subscription



👉 Find me on [LinkedIn](#) | [Twitter](#) | [Threads](#) | [Instagram](#)

Thank you for supporting this newsletter.

You are now 200,001+ readers strong, very close to 201k. Let's try to get 201k readers by 5 February. Consider sharing this post with your friends and get rewards.

Y'all are the best.

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share

-
- 1 A segment is a short piece of a video (usually a few seconds long) that the full video is split into so it can be streamed smoothly and switch quality on the fly.
 - 2 Daily Active Users: a metric used to measure the number of unique users who engage with the product in a single day.
 - 3
 - Sharding means splitting data across many machines so no single machine gets overloaded.
 - Partitioning means deciding how that data is divided (for example, by video ID or user ID).
 - Caching means keeping frequently used data in fast memory so it can be served quickly without hitting the database.

- 4
 - Multipart upload means sending a large file in many smaller pieces instead of all at once, so failures can be retried without restarting the whole upload.
 - Chunking means breaking the video into small parts (for upload or streaming), so system can process, store, and stream it efficiently.
- 5
 - 4K/8K: Refers to Ultra High Definition video resolutions (approx. 4,000 and 8,000 pixels wide, respectively).
 - AVI: Audio Video Interleave; a multimedia container format introduced by Microsoft.
 - MOV: A multimedia container file format developed by Apple (QuickTime).
 - MP4: MPEG-4 Part 14; a digital multimedia container format commonly used to store video and audio.
- 6 Renditions means different versions of the same video, each encoded at a different resolution or quality (for example, 4K, 1080p, 720p), so it can play well on different devices and internet speeds.
- 7 Content Delivery Network: a geographically distributed network of proxy servers and their data centers used to provide high availability and performance.
- 8 Adaptive streaming is a way of playing video where the quality automatically adjusts while you watch, based on how fast & stable your internet connection is, so the video keeps playing without buffering.
- 9 Consistency, Availability, Partition Tolerance: a theorem stating that a distributed data store can effectively provide only two of the three guarantees.
- 10
 - Redundancy means having extra copies of systems or data, so if one part fails, another can take over.
 - Failover means automatically switching to those backups when a failure happens, so the service keeps running.
- 11 It means saving the original uploaded video file so the system can later convert it to different qualities and versions during transcoding. Transcoding means

converting a video from its original format into multiple versions with different qualities or resolutions, so it can play smoothly on different devices and internet speeds.

- 12 Representational State Transfer is a software architectural style often used in web services development.
- 13 Temporary, signed URL for direct uploads to cloud storage
- 14 Cursor-based pagination is a way to load data page by page using a pointer (cursor) to the last item seen, rather than page numbers, so it stays fast and consistent even when new data is added.
- 15
 - Metadata is small, descriptive information about a video, not the video itself. It's lightweight, frequently read, and stored in databases.
 - Video data is the actual heavy binary content that users watch. It is huge, immutable, and bandwidth-intensive.
- 16 The manifest file URL points to a small text file that tells the video player where all the video pieces are and which qualities are available. It does not contain the video itself. It contains a list of segment URLs for different qualities (4K, 1080p, 720p, etc)
- 17 DynamoDB is a fully managed cloud database by AWS that stores data as key-value pairs, designed to handle very large scale and extremely high request rates with low latency.
- 18 A search index is a specialized data structure that enables the system to quickly find videos based on keywords or filters, without scanning the entire database.
- 19 The “skip 10 million rows” problem happens when a database has to scan and discard a huge number of rows just to reach the page you asked for.

With offset-based pagination, a query like “give me page 1,000,001” forces the database to:

- Read the first 10,000,000 rows

- Throw them away
- Then return the next few rows

...this becomes very slow and expensive at scale.

Cursor-based pagination avoids this by remembering where you left off and continuing from there, so the database doesn't need to skip millions of rows.

- 20 It makes queries much faster because the database doesn't have to contact other machines over the network. It also reduces system load and complexity, since all the needed data is already in one place.
- 21 Blob storage is a type of cloud storage designed to store very large, unstructured files like videos, images, audio, and backups:
- It stores data as files (objects), not rows and tables
 - It scales to huge sizes (terabytes to exabytes)
 - It's ideal for things like video files, photos, and raw uploads, not database queries

Examples are Amazon S3, Google Cloud Storage, and Azure Blob Storage.

- 22 An .m3u8 playlist is a small text file used in video streaming that tells the video player which video segments to download and in what order.
- 23 The database automatically adds capacity as write load increases.
- 24 Thrashing occurs when the database is overwhelmed by too many small, frequent updates, spending most of its time handling writes rather than doing useful work.
- 25 Scatter-gather is a query pattern in which the system sends the same request to multiple database shards in parallel (scatter) and then collects and merges the results (gather) before returning them to the user.
- 26 A long-tail video is a video that very few people watch, so it isn't requested often enough to stay cached on CDN servers...

- Popular videos get watched a lot and stay in the CDN cache
- Long-tail videos are rarely watched and usually have to be fetched from the origin storage the first time someone plays them

This is why long-tail videos may start slightly slower at first, but caching them afterward helps future viewers.

- 27 Apache Airflow and AWS Step Functions are workflow tools that coordinate and run multi-step jobs in the correct order, automatically handling retries, parallel execution, and failures.
- 28 Digital Rights Management: technologies used to control the use of proprietary hardware and copyrighted works.
- 29 FFmpeg is a popular open-source tool that converts, compresses, and processes audio and video files, making it the standard engine used for video transcoding.
- 30 A spot instance is a discounted cloud server that uses spare capacity at a much lower price, with the tradeoff that it can be shut down, which is acceptable for retryable jobs like video transcoding.
- 31 A cache hit means the requested video is already stored in the CDN cache, so it can be served immediately without contacting the origin.
- 32 A cache miss means the video is not in the cache, so the CDN fetches it from the origin storage before serving it to the user (and then caches it for future requests).
- 33
 - ISP (Internet Service Provider) is the company that gives people internet access, like the network that connects your home or phone to the internet.
 - IXP (Internet Exchange Point) is a place where many internet networks connect directly to each other, enabling data to be exchanged faster and more cheaply without going through long, indirect routes.
- 34 Cold start problem occurs when content is not yet cached anywhere, so the first few users have to wait while the system fetches it from the origin storage,

resulting in higher latency. Once the content is fetched and cached, subsequent users get faster responses, but the first viewers experience slower playback.

- 35 A heartbeat is a regular “I’m alive” signal that a service sends to show it’s still running and healthy. If the heartbeat stops or looks unhealthy, the system assumes the service has failed, and the load balancer stops sending traffic to it, keeping the system available.
- 36 Exponential backoff is a retry strategy in which each failed attempt waits progressively longer before retrying.
- First retry after 1 second
 - Next retry after 2 seconds
 - Then 4 seconds, then 8 seconds

This prevents many clients from retrying at the same time and overwhelming the system while it’s recovering.

- 37 Checkpointing means saving partial progress while a job is running, so if it fails, it can restart from the last saved point instead of starting over from the beginning.